

Segunda Lista de Exercícios - Programação Concorrente

NOME Davi dos Santos Mattos **DRE:** 119133049

Prof. Gabriel P. Silva - 2026.1

1) Qual a diferença entre o valor retornado pelas seguintes rotinas dentro e fora de uma região paralela? (0,5 ponto)

a) `omp_get_num_threads()`

R: Retorna o número de threads que estão atualmente ativas. Fora da região paralela retorna 1, pois apenas a thread principal está executando. Dentro da região paralela retorna a quantidade de threads ativas no grupo criador por `#pragma omp parallel`.

b) `omp_get_max_threads()`

R: Retorna o número máximo de threads que o OpenMP disponibilizará caso uma região paralela seja iniciada logo em seguida sem especificar um número exato.

Fora da região: Retorna o valor máximo padrão do sistema (geralmente igual ao número de núcleos do seu processador).

Dentro da região: Geralmente retorna o mesmo valor máximo configurado para o ambiente, pois é uma consulta de capacidade, não do estado atual.

c) `omp_get_num_procs()`

R: Retorna o número de processadores (núcleos lógicos) físicos disponíveis na máquina onde o código está rodando.

Fora e Dentro da região retorna exatamente o mesmo valor.

d) `omp_in_parallel()`

R: Retorna um valor booleano (verdadeiro ou falso) indicando se o código atual está sendo executado por múltiplas threads em paralelo. Fora da região retorna 0 (Falso). Dentro da região retorna 1 (Verdadeiro).

2) Quais as diferenças entre a declaração de variáveis `private`, `firstprivate` e `lastprivate`? Em quais diretivas são utilizadas? (0,5 ponto)

R:

- `private` cria uma nova instância da variável para cada thread. Pode ser utilizada nas diretivas `parallel`, `for`, `sections`, e `task`
- `firstprivate` cada thread cria sua própria cópia da variável, mas essa cópia é inicializada com o valor que a variável tinha antes de entrar na região paralela. Pode ser utilizada nas diretivas `parallel`, `for`, `sections`, `single`, e `task`

- `lastprivate` garante que, ao sair da região, a variável original (fora) receba o valor da variável correspondente da última iteração do laço. Pode ser utilizadas com as diretivas `for`, `sections` (e as formas combinadas `parallel for`, `parallel sections`) e `taskloop`.

3) Quais as diferenças entre as diretivas `master` e `single`? (0,5 ponto)

R: `master` garante que a única thread a executar o bloco seja sempre a thread mestre (ID 0) e não possui uma barreira implícita, enquanto `single`, garante que a única thread a executar o bloco seja a primeira que chegar e possui uma barreira implícita

4) Forneça um exemplo do uso da cláusula `reduction` no OpenMP. Descreva quatro tipos de operações de redução que podem ser utilizadas com esta cláusula. (0,5 ponto)

R: Soma de elementos em um vetor

```
#include <stdio.h>
#include <omp.h>

int main() {
    int i, n = 1000;
    int soma = 0;
    int vetor[1000];

    for(i = 0; i < n; i++) vetor[i] = 1; // Inicializando vetor

    #pragma omp parallel for reduction(+:soma)
    for(i = 0; i < n; i++) {
        soma += vetor[i];
    }

    printf("Resultado total da soma: %d\n", soma);
    return 0;
}
```

Quatro tipos comuns de operações de redução:

Soma (+): Acumula a soma dos elementos.

Produto (*): Acumula o produto dos elementos.

Mínimo (min): Encontra o menor valor entre todos os elementos.

Máximo (max): Encontra o maior valor entre todos os elementos.

5) No código abaixo, faça um diagrama do escalonamento das iterações do laço entre 4 threads. (0,5 ponto)

```
#pragma omp for schedule(guided,5)
for (j = 0; j < 50; j++)
```

```
a[j] = b[j] + c[j];
```

R:

Iterações restantes	Tamanho	j	Thread
50	13	0-12	A
37	10	13-22	B
27	7	23-29	C
20	5	30-34	D
15	5	35-39	A
10	5	40-44	B
5	5	45-49	C

6) Reescreva o código seguinte, colocando as diretivas `barrier` explicitamente, de modo a maximizar o desempenho, mas sem alterar o resultado da execução. (0,5 ponto)

R:

```
#pragma omp parallel
{
    #pragma omp for nowait
    for (j = 0; j < n; j++)
        a[j] = b[j] + c[j];

    #pragma omp for nowait
    for (j = 0; j < n; j++)
        d[j] = e[j] * f;

    #pragma omp barrier

    #pragma omp for nowait
    for (j = 0; j < n; j++) {
        z[j] = (a[j] + a[j+1]) * 0.5;
    }
}
```

7) Paralelizar o exemplo a seguir sem o uso da diretiva `for` ou `parallel for`. (0,5 ponto)

R:

```
#pragma omp parallel
{
    int id = omp_get_thread_num();
    int nthreads = omp_get_num_threads();
```

```

int i;

for (i = id; i < n; i += nthreads)
    z[i] = a * x[i] + y;
}

```

8) Explique o porquê cada um dos laços a seguir pode ou não pode ser paralelizado com a diretiva `parallel for`. (0,5 ponto)

a)

```

for (i = 0; i < N; i++)
    if (x[i] > maxval) break;

```

R: Não pode o comando `break` exige que a verificação seja feita de forma sequencial, se uma thread encontrar o `maxval` primeiro, ela deve parar o laço inteiro, o que é complexo de sincronizar paralelamente.

b)

```

for (i = 0; i < N; i++)
    for (j = 0; j < i; j++)
        a[j][i] = a[j+1][i];

```

R: O laço externo pode ser paralelizado: cada valor de `i` opera sobre uma coluna `i` própria, e colunas diferentes nunca se tocam. Mas o laço interno (`j`) não, pois tem uma dependência: o valor lido em `a[j+1][i]` na iteração `j` é o mesmo índice que será escrito na iteração `j+1`. Se as iterações de `j` rodarem fora de ordem, uma thread pode escrever em `a[j+1][i]` antes que outra leia esse valor.

c)

```

for (k = 0; k < N; k++)
    x[k] = q + y[k] * (r * z[k+10] + t * z[k+11]);

```

R: **Sim.** Cada iteração `k` escreve apenas em `x[k]` e lê de `y[k]` e `z[k+10]`. Como nenhuma escrita sobrepõe uma leitura de outra iteração (a escrita é sempre em `x[k]`), não há dependência de dados.

d)

```

for (i = 1; i < N; i++)
    x[i] = z[i] * (y[i] - x[i-1]);

```

R: Não pode. `x[i]` depende explicitamente de `x[i-1]`, que foi calculado na iteração anterior, é uma recorrência sequencial. Cada iteração só pode rodar depois que a anterior terminou.

e)

```
for (i = 0; i < N; i++) {
    if (fabs(a[i]) > machine_max || fabs(a[i]) < machine_min) {
        printf("i=%d \n", i);
        break;
    }
    a[i] = a[i] * a[i];
}
```

R: **Não**. Assim como no item (a), o comando `break` exige que a verificação seja feita de forma sequencial.

f)

```
for (i = 1; i < N; i++)
    for (k = 0; k < i; k++)
        w[i] += b[k][i] * w[(i-k)-1];
```

R: **Não**. O valor de `w[i]` depende de valores anteriores de `w`.

g)

```
for (k = 0; k < N; k++)
    x[k] = u[k] + r * (z[k] + r * y[k]) +
        t * (u[k+3] + r * (u[k+2] + r * u[k+1])) +
        t * (u[k+6] + r * (u[k+5] + r * u[k+4]));
```

R: Sim, todos os índices acessados (`u[k]`, `z[k]`, `u[k+3]`, etc.) são constantes ou dependem apenas do índice k atual. Não há escrita em um índice que será lido por uma iteração futura de forma conflituosa.

9) Considere o seguinte laço: (0,5 ponto)

```
x = 1;
#pragma omp parallel for firstprivate(x)
for(i = 0; i < N; i++) {
    y[i] = x + i;
    x = i;
}
```

a) Porque este laço está incorreto? `y[i]` recebe o mesmo resultado independente do número de threads executando o laço?

R: Por causa de `firstprivate(x)` que garante que o valor de `x` dentro da região paralela será sempre 1. Não, cada thread reinicia sua cópia de `x` em 1, então só o primeiro `i` de cada bloco/thread sai errado.

b) Qual o valor da variável `i` ao final do laço? Qual é o valor da variável `x` ao final do laço?

R: `i` = indefinido. `x` = 1

c) Qual seria o valor de `x` ao final do laço se seu escopo fosse `shared` ?

R: Não tem como saber, pois se `x` fosse `shared` , todas as threads leriam e escreveriam na **mesma posição de memória**, simultaneamente, sem nenhuma sincronização. Isso é uma **condição de corrida** clássica: não existe garantia de ordem entre as escritas de diferentes threads.

d) Este laço pode ser paralelizado corretamente (isto é, preservando a semântica sequencial) apenas com o uso de diretivas OpenMP?

R: Não, pois a lógica `y[i] = x + i` combinada com `x = i` estabelece uma dependência de dados.

10) Quais os mecanismos disponíveis no OpenMP para lidar com as condições de corrida? (0,5 ponto)

R:

- `atomic` - garante que uma operação específica de leitura e escrita em uma memória seja executada de forma atômica
- `critical` - cria uma "seção crítica". Apenas uma thread por vez pode entrar nesse bloco de código.
- `barrier` - força todas as threads a pararem e esperarem até que todo o grupo chegue ao mesmo ponto.
- `lock` - bloqueiam e desbloqueiam as threads
- `reduction` - deixa o OpenMP gerenciar cópias privadas + combinação final automaticamente, sem você precisar de bloquear nenhuma thread.

11) Descreva como a diretiva `ordered` é utilizada no OpenMP. Apresente um trecho de código como exemplo. (0,5 ponto)

R: `ordered` é tanto uma cláusula da diretiva `for` , quanto uma diretiva por si só, que serve para indicar que o trecho deve ser executado como se fosse sequencialmente.

```
#include <stdio.h>
#include <omp.h>
```

```

int main() {
    int i;

    #pragma omp parallel for ordered
    for(i = 0; i < 5; i++) {

        int valor = algum_calculo(i);

        #pragma omp ordered
        {
            printf("Iteração %d processada: %d\n", i, valor);
        }
    }
    return 0;
}

```

12) Como a cláusula `collapse` funciona no OpenMP e em que situações deve-se utilizá-la? (0,5 ponto)

R: A cláusula `collapse` serve para paralelizar laços de repetições aninhados, fundindo as iterações, pois sem esta cláusula, o a diretiva `for` só "enxerga" o laço de repetição externo. `collapse` é utilizado quando o laço externo não tiver repetições o suficiente para ocupar todas as threads.

13) Explique a diferença fundamental entre tarefas explícitas e tarefas implícitas no OpenMP. Em que contextos cada tipo de tarefa é gerado? (0,5 ponto)

R: Tarefas implícitas são as que são geradas automaticamente pelo ambiente ao entrar em uma região paralela (`pragma omp parallel`), enquanto tarefas explícitas são aquelas que são definidas manualmente através de `pragma omp task`

14) Defina o que é um ponto de escalonamento de tarefas. Liste pelo menos três ações distintas que uma thread pode tomar ao encontrar um ponto de escalonamento durante a execução de uma tarefa. (0,5 ponto)

R: Um ponto de escalonamento de tarefas é um lugar específico, dentro da execução de uma tarefa, onde a thread que está executando aquela tarefa tem permissão do runtime para parar o que está fazendo e considerar fazer outra coisa, podendo ser, começar uma tarefa nova, retomar uma que tinha pausado antes, ou simplesmente continuar de onde estava.

15) O que são os pontos de sincronização de tarefas, e como a diretiva `#pragma omp taskwait` é especificamente utilizada neste contexto? (0,5 ponto)

R: Um ponto de sincronização de tarefas é um lugar no código onde se estabelece uma garantia de conclusão: a execução só pode seguir adiante depois que determinadas tarefas

forem concluídas. `taskwait` é uma diretiva que suspende a execução da tarefa atual e faz com que a thread aguarde até que todas as tarefas filhas diretas sejam concluídas.

16) No trecho de código a seguir, identifique adequadamente os pontos de escalonamento e de sincronização de código. (1,0 ponto)

```
#include <stdio.h>
#include <omp.h>

int main() {
    #pragma omp parallel num_threads(4)
    {
        #pragma omp single
        {
            #pragma omp task // <----- ESCALONAMENTO
            printf("Executando Tarefa A \n");

            // <----- ESCALONAMENTO

            #pragma omp taskyield // <----- ESCALONAMENTO
            printf("Após taskyield \n"); //

            #pragma omp task // <----- ESCALONAMENTO
            printf("Executando Tarefa B \n");

            // <----- ESCALONAMENTO

            #pragma omp taskwait // <----- SINCRONIZAÇÃO E ESCALONAMENTO
            printf("Após taskwait \n");

            int data = 0;

            #pragma omp task depend(out: data) // <----- ESCALONAMENTO
            data = 42;

            #pragma omp task depend(in: data) // <----- ESCALONAMENTO
            printf("Data = %d\n", data);

            #pragma omp taskgroup
            {
                #pragma omp task // <----- ESCALONAMENTO
                printf("Tarefa E no taskgroup \n");

                #pragma omp task // <----- ESCALONAMENTO
                printf("Tarefa F no taskgroup \n");
            } // Fim do taskgroup
        } // Fim do single
    } // Fim do parallel
}
```



```
    return 0;
} // Fim do programa
```

R:

17) Descreva a função das cláusulas `if` e `final` e como elas se diferenciam na geração e execução das tarefas. (0,5 ponto)

R: A cláusula `if` controla a criação da tarefa baseada em uma condição booleana, se a condição for verdadeira, a tarefa é criada e pode ser executada por qualquer thread. Se for falsa, a tarefa é executada imediatamente pela thread atual.

A cláusula `final` força uma tarefa a não gerar mais tarefas filhas, se a condição da cláusula `final` for verdadeira, a tarefa e todas as suas tarefas descendentes devem ser executadas imediatamente pela thread atual, sem que nenhuma nova tarefa seja criada ou enfileirada.

18) Considere o seguinte trecho de código. Aplique a cláusula `depend` com os argumentos corretos para garantir a execução correta das tarefas. Em seguida, desenhe o grafo de dependências. (0,5 ponto)

```
int main() {
    int A = 0, B = 0, C = 0;

    #pragma omp parallel
    {
        #pragma omp single
        {
            #pragma omp task depend(out:A)
            // Tarefa A
            A = 1;

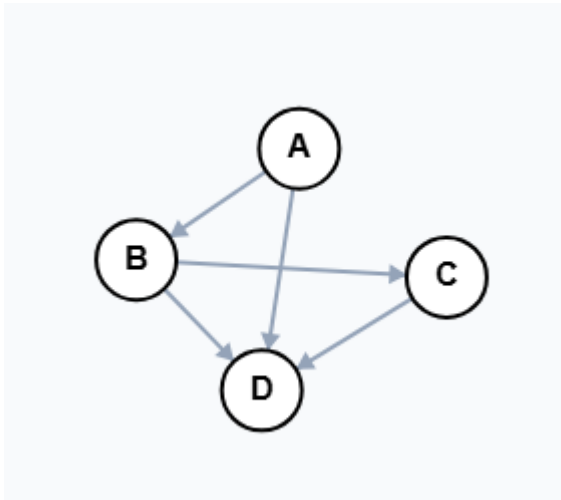
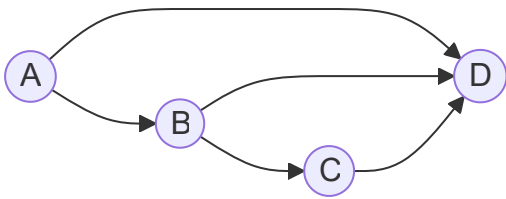
            #pragma omp task depend(in: A) depend(out: B)
            // Tarefa B
            B = A + 1;

            #pragma omp task depend(in:B) depend(inout:C)
            // Tarefa C
            C = B + C;

            #pragma omp task depend(in:A,B,C)
            // Tarefa D
            printf("Resultado final %d %d %d \n", A, B, C);
        } // Fim do single
    } // Fim do parallel

    return 0;
} // Fim do programa principal
```

R:



19) Desenhe o grafo de dependências para o trecho de código a seguir: (1,0 ponto)

```
int main() {  
    int x = 0, y = 8, z = 0;  
  
    #pragma omp parallel  
    {  
        #pragma omp single  
        {  
            // Task 1:  
            #pragma omp task depend(out: x) shared(x)  
            x = 5;  
  
            // Task 2:  
            #pragma omp task depend(in: x, y, z) shared(x, y, z)  
            int resultado = x + y + z;  
  
            // Task 3:  
            #pragma omp task depend(in: x) depend(out: y) shared(x, y)  
            y = x * 2;  
  
            // Task 4:  
            #pragma omp task depend(in: x) depend(out: z) shared(x, z)  
            z = x + 10;  
  
            // Task 5:  
            #pragma omp task depend(inout: y) shared(y)  
            y = y + 3;  
        }  
    }  
}
```

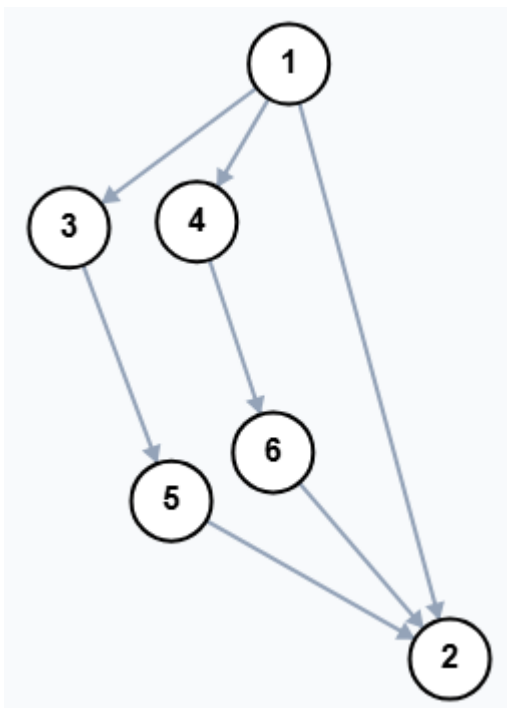
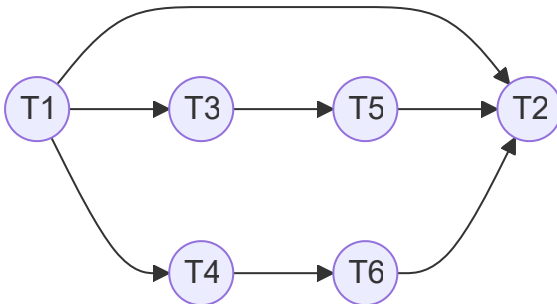
```

// Task 6:
#pragma omp task depend(inout: z) shared(z)
z = z - 2;
}
}

printf("Valores finais: x=%d, y=%d, z=%d\n", x, y, z);
return 0;
}

```

R:



20) Explique a diretiva `#pragma omp taskloop`. Mencione pelo menos duas cláusulas específicas que podem ser usadas com `taskloop` e descreva suas funções. (0,5 ponto)

R: A diretiva `taskloop` transforma um laço de repetição em pequenos blocos de tarefa (task). Uma de suas cláusulas são, `grainsize(n)`, que especifica o número aproximado de iterações que cada tarefa gerada deve conter, `num_tasks(n)`: especifica diretamente o número de tarefas a serem criadas.